

Evaluation

Technically, there are very few steps to run it on GPUs, elsewhere (ie. on Lamini).

```
finetuned_model = BasicModelRunner(  
    "lamini/lamini_docs_finetuned"  
)  
finetuned_output = finetuned_model(  
    test_dataset_list # batched!  
)
```

Let's look again under the hood! This is the open core code of Lamini's llama library :)

```
In [2]: ▶ import datasets  
import tempfile  
import logging  
import random  
import config  
import os  
import yaml  
import logging  
import difflib  
import pandas as pd  
  
import transformers  
import datasets  
import torch  
  
from tqdm import tqdm  
from utilities import *  
from transformers import AutoTokenizer, AutoModelForCausalLM  
  
logger = logging.getLogger(__name__)  
global_config = None
```

```
In [3]: dataset = datasets.load_dataset("lamini/lamini_docs")
        test_dataset = dataset["test"]
```

```
Downloading readme: 0%|          | 0.00/577 [00:00<?, ?B/s]
Downloading data files: 0%|        | 0/2 [00:00<?, ?it/s]
Downloading data: 0%|            | 0.00/615k [00:00<?, ?B/s]
Downloading data: 0%|            | 0.00/83.7k [00:00<?, ?B/s]
Extracting data files: 0%|         | 0/2 [00:00<?, ?it/s]
Generating train split: 0%|         | 0/1260 [00:00<?, ? examples/s]
Generating test split: 0%|          | 0/140 [00:00<?, ? examples/s]
```

```
In [4]: print(test_dataset[0]["question"])
        print(test_dataset[0]["answer"])
```

Can Lamini generate technical documentation or user manuals for software projects?

Yes, Lamini can generate technical documentation and user manuals for software projects. It uses natural language generation techniques to create clear and concise documentation that is easy to understand for both technical and non-technical users. This can save developers a significant amount of time and effort in creating documentation, allowing them to focus on other aspects of their projects.

```
In [5]: model_name = "lamini/lamini_docs_finetuned"
        tokenizer = AutoTokenizer.from_pretrained(model_name)
        model = AutoModelForCausalLM.from_pretrained(model_name)
```

```
/usr/local/lib/python3.9/site-packages/huggingface_hub/file_download.py:943: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
```

```
tokenizer_config.json: 0%|          | 0.00/264 [00:00<?, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0%|         | 0.00/99.0 [00:00<?, ?B/s]
```

Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.

```
config.json: 0%|                | 0.00/717 [00:00<?, ?B/s]
model.safetensors: 0%|           | 0.00/282M [00:00<?, ?B/s]
generation_config.json: 0%|        | 0.00/111 [00:00<?, ?B/s]
```

Setup a really basic evaluation function

```
In [6]: ▶ def is_exact_match(a, b):  
        return a.strip() == b.strip()
```

```
In [7]: ▶ model.eval()
```

```
GPTNeoXForCausalLM(  
  (gpt_neox): GPTNeoXModel(  
    (embed_in): Embedding(50304, 512)  
    (emb_dropout): Dropout(p=0.0, inplace=False)  
    (layers): ModuleList(  
      (0-5): 6 x GPTNeoXLayer(  
        (input_layernorm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)  
        (post_attention_layernorm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)  
        (post_attention_dropout): Dropout(p=0.0, inplace=False)  
        (post_mlp_dropout): Dropout(p=0.0, inplace=False)  
        (attention): GPTNeoXAttention(  
          (rotary_emb): GPTNeoXRotaryEmbedding()  
          (query_key_value): Linear(in_features=512, out_features=1536, bias=True)  
          (dense): Linear(in_features=512, out_features=512, bias=True)  
          (attention_dropout): Dropout(p=0.0, inplace=False)  
        )  
        (mlp): GPTNeoXMLP(  
          (dense_h_to_4h): Linear(in_features=512, out_features=2048, bias=True)  
          (dense_4h_to_h): Linear(in_features=2048, out_features=512, bias=True)  
          (act): GELUActivation()  
        )  
      )  
    )  
    (final_layer_norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)  
    (embed_out): Linear(in_features=512, out_features=50304, bias=False)  
  )  
)
```

```
In [8]: ▶ def inference(text, model, tokenizer, max_input_tokens=1000, max_output_tokens=1000):
# Tokenize
tokenizer.pad_token = tokenizer.eos_token
input_ids = tokenizer.encode(
    text,
    return_tensors="pt",
    truncation=True,
    max_length=max_input_tokens
)

# Generate
device = model.device
generated_tokens_with_prompt = model.generate(
    input_ids=input_ids.to(device),
    max_length=max_output_tokens
)

# Decode
generated_text_with_prompt = tokenizer.batch_decode(generated_tokens_with_prompt)

# Strip the prompt
generated_text_answer = generated_text_with_prompt[0][len(text):]

return generated_text_answer
```

Run model and compare to expected answer

```
In [9]: ▶ test_question = test_dataset[0]["question"]
generated_answer = inference(test_question, model, tokenizer)
print(test_question)
print(generated_answer)
```

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

Can Lamini generate technical documentation or user manuals for software projects?

Yes, Lamini can generate technical documentation or user manuals for software projects. This can be achieved by providing a prompt for a specific technical question or question to the LLM Engine, or by providing a prompt for a specific technical question or question. Additionally, Lamini can be trained on specific technical questions or questions to help users understand the process and provide feedback to the LLM Engine. Additionally, Lamini

```
In [10]: ▶ answer = test_dataset[0]["answer"]  
         print(answer)
```

Yes, Lamini can generate technical documentation and user manuals for software projects. It uses natural language generation techniques to create clear and concise documentation that is easy to understand for both technical and non-technical users. This can save developers a significant amount of time and effort in creating documentation, allowing them to focus on other aspects of their projects.

```
In [11]: ▶ exact_match = is_exact_match(generated_answer, answer)  
         print(exact_match)
```

False

Run over entire dataset

```
In [12]: n = 10
metrics = {'exact_matches': []}
predictions = []
for i, item in tqdm(enumerate(test_dataset)):
    print("i Evaluating: " + str(item))
    question = item['question']
    answer = item['answer']

    try:
        predicted_answer = inference(question, model, tokenizer)
    except:
        continue
    predictions.append([predicted_answer, answer])

    #fixed: exact_match = is_exact_match(generated_answer, answer)
    exact_match = is_exact_match(predicted_answer, answer)
    metrics['exact_matches'].append(exact_match)

    if i > n and n != -1:
        break
print('Number of exact matches: ', sum(metrics['exact_matches']))
```

0it [00:00, ?it/s]The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention mask` to obtain reliable results.

Setting `pad token id` to `eos token id`:0 for open-end generation.

[illegible]

```
In [13]: ► df = pd.DataFrame(predictions, columns=["predicted_answer", "target_answer"]
print(df)
```

```

                                predicted_answer \
0  Yes, Lamini can generate technical documentati...
1  You can use the Authorization HTTP header to g...
2  Lamini's LLM Engine is a LLM Engine for develo...
3  Yes, there is a community or support forum ava...
4  Yes, the Lamini library can be utilized for te...
5  Lamini is designed to be flexible and scalable...
6  You can instantiate the LLM engine using the L...
7  Yes, Lamini provides mechanisms for compressio...
8  The performance of LLMs trained using Lamini c...
9  Yes, there is a support and community availabl...
10 Yes, there are some code samples available in ...
11 Yes, there are some code samples available tha...
```

```

                                target_answer
0  Yes, Lamini can generate technical documentati...
1  The Authorization HTTP header should include t...
2  Yes, the code includes a version parameter in ...
3  Yes, there is a community forum available for ...
4  The Lamini library is not specifically designe...
5  Lamini offers both free and paid plans for usi...
6  You can instantiate the LLM engine using the l...
7  Yes, Lamini provides mechanisms for model comp...
8  According to the information provided, Lamini ...
9  Yes, there is a support community available to...
10 Yes, the Python logging module documentation p...
11 Yes, there is a separate section in the docume...
```

Evaluate all the data

```
In [14]: ► evaluation_dataset_path = "lamini/lamini_docs_evaluation"
evaluation_dataset = datasets.load_dataset(evaluation_dataset_path)
```

```
Downloading readme: 0%|          | 0.00/413 [00:00<?, ?B/s]
```

```
Downloading data files: 0%|          | 0/1 [00:00<?, ?it/s]
```

```
Downloading data: 0%|          | 0.00/86.1k [00:00<?, ?B/s]
```

```
Extracting data files: 0%|          | 0/1 [00:00<?, ?it/s]
```

```
Generating train split: 0%|          | 0/139 [00:00<?, ? examples/s]
```

```
In [15]: ▶ pd.DataFrame(evaluation_dataset)
```

	train
0	{'predicted_answer': 'Yes, Lamini can generate...
1	{'predicted_answer': 'You can use the Authoriz...
2	{'predicted_answer': 'Lamini's LLM Engine is a...
3	{'predicted_answer': 'Yes, there is a communit...
4	{'predicted_answer': 'Yes, the Lamini library ...
...	...
134	{'predicted_answer': 'Yes, Lamini has the abil...
135	{'predicted_answer': 'I wish! This documentati...
136	{'predicted_answer': 'Yes, Lamini can generate...
137	{'predicted_answer': 'Yes, the documentation h...
138	{'predicted_answer': 'Yes, you can learn to us...

139 rows × 1 columns

Try the ARC benchmark

This can take several minutes


```
In [16]: ▶ !python lm-evaluation-harness/main.py --model hf-causal --model_args pretr
```

```
huggingface/tokenizers: The current process just got forked, after parallelism  
has already been used. Disabling parallelism to avoid deadlocks...
```

```
To disable this warning, you can either:
```

- Avoid using `tokenizers` before the fork if possible
- Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true
| false)

```

Selected Tasks: ['arc_easy']
Using device 'cpu'
/usr/local/lib/python3.9/site-packages/huggingface_hub/file_download.py:943: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.
Downloading readme: 9.00kB [00:00, 8.46MB/s]
Downloading data files: 0%|                               | 0/3 [00:00<?, ?it/s]
Downloading data: 0%|                                   | 0.00/331k [00:00<?, ?B/s]
Downloading data: 100%|██████████████████████████████| 331k/331k [00:00<00:00, 605kB/s]
Downloading data files: 33%|██████████                 | 1/3 [00:00<00:01, 1.82it/s]
Downloading data: 0%|                                   | 0.00/346k [00:00<?, ?B/s]
Downloading data: 100%|██████████████████████████████| 346k/346k [00:00<00:00, 693kB/s]
Downloading data files: 67%|██████████████████         | 2/3 [00:01<00:00, 1.92it/s]
Downloading data: 0%|                                   | 0.00/86.1k [00:00<?, ?B/s]
Downloading data: 100%|██████████████████████████████| 86.1k/86.1k [00:00<00:00, 439kB/s]
Downloading data files: 100%|██████████████████████████| 3/3 [00:01<00:00, 2.40it/s]
Extracting data files: 100%|██████████████████████████| 3/3 [00:00<00:00, 3005.95it/s]
Generating train split: 100%|██████                    | 2251/2251 [00:00<00:00, 428588.60 examples/s]
Generating test split: 100%|██████                     | 2376/2376 [00:00<00:00, 672628.67 examples/s]
Generating validation split: 100%|████                 | 570/570 [00:00<00:00, 344836.76 examples/s]
Task: arc_easy; number of docs: 2376
Task: arc_easy; document 0; context prompt (starting on next line):
Question: Which is the function of the gallbladder?
Answer:
(end of prompt on previous line)
Requests: [Req_loglikelihood('Question: Which is the function of the gallbladder?\nAnswer:', 'store bile')[0],
Req_loglikelihood('Question: Which is the function of the gallbladder?\nAnswer:', 'produce bile')[0],
Req_loglikelihood('Question: Which is the function of the gallbladder?\nAnswer:', 'store digestive enzymes')[0],
Req_loglikelihood('Question: Which is the function of the gallbladder?\nAnswer:', 'produce digestive enzymes')[0]]
Running loglikelihood requests
100%|██████████████████████████████████████████████████| 9496/9496 [03:35<00:00, 44.12it/s]
{
  "results": {

```

```

    "arc_easy": {
        "acc": 0.3122895622895623,
        "acc_stderr": 0.009509325983631455,
        "acc_norm": 0.3135521885521885,
        "acc_norm_stderr": 0.009519779157242258
    }
},
"versions": {
    "arc_easy": 0
},
"config": {
    "model": "hf-causal",
    "model_args": "pretrained=lamini/lamini_docs_finetuned",
    "num_fewshot": 0,
    "batch_size": null,
    "batch_sizes": [],
    "device": "cpu",
    "no_cache": false,
    "limit": null,
    "bootstrap_iters": 100000,
    "description_dict": {}
}
}
hf-causal (pretrained=lamini/lamini_docs_finetuned), limit: None, provide_desc
ription: False, num_fewshot: 0, batch_size: None
| Task | Version | Metric | Value | Stderr |
|-----|-----|-----|-----|-----|
| arc_easy | 0 | acc | 0.3123 | ± 0.0095 |
| | | acc_norm | 0.3136 | ± 0.0095 |

```